

Разработка и анализ требований

Лекция №3.3

Требования в управлении проектом



Содержание лекции

- Требования к созданию АИС.
- Оценка целесообразности создания АИС.
- Планирование проекта на основе требований.
- Путь гибких Agile методологий.
- Стратегии работы по управлению рисками.
- Выбор в отношении разработки или покупки ПО.
- Стратегии выбора.

Требования к созданию АИС

- *для того, чтобы создать АИС, сначала нужно проанализировать возможность создания.*
- Где найти Заказчика, который согласится ждать 2-3 месяца, пока Разработчик составит для него коммерческое предложение?
- Кто оплатит работы по анализу требований?
- Как быть, если цена вопроса окажется непомерной и от проекта придётся отказаться – кто возместит Заказчику убытки на проведение исследований?

Оценка целесообразности создания АИС

- Выделить 25 – 99 функций, характеризующих систему;
- Установить приоритеты для каждой из функций;
- Оценить трудозатраты;
- Оценить риски;
- Свести оценки в таблицу:

№ пп	Функция	Приоритет	Трудоемкость	Риск

Планирование проекта на основе требований, путь RUP

Исходя из базовой концепции планирования итерационных проектов, план проекта разбивается на фазы:

- Начало,
 - Уточнение,
 - Конструирование,
 - Переход.
-
- Назначаются даты окончания каждой фазы.
 - Назначаются даты окончания каждой малой итерации.

Планирование проекта на основе требований, путь RUP

1. Укрупнённый план работ составляется «как можно раньше».
2. Бюджет появляется лишь к окончанию первой фазы (она может длиться месяц, а может и полгода).
3. Как план, так и бюджет проекта представляют собой лишь прогноз, который может корректироваться на протяжении работ над проектом.
4. На момент появления плана и бюджета должно появиться подробное описание лишь 20% ключевой функциональности системы и «широкое, но неглубокое» описание 80% функциональности.

План итерации

Более конкретная информация представлена в **плане итерации**. Основные его особенности:

1. План итерации базируется на функциональных требованиях.
2. План итерации имеет жёсткие сроки.
3. Точный план составляется на одну, очередную итерацию.

Требования в гибких методологиях

Манифест гибкой разработки:

- Индивидуальности и взаимодействия ВЫШЕ процессов и инструментов;
- работающее программное обеспечение ВЫШЕ всесторонней документации;
- сотрудничество с клиентами ВЫШЕ переговоров по контракту;
- реакция на изменения ВЫШЕ следования плану.

и 12 приложений (в столь же лаконичной форме) к нему

www.agilealliance.org

Agile методологии

Примерами agile-методологий являются

- *Adaptive Software Development (ASD),*
- *Lean Development,*
- *SCRUM,*
- *Kanban,*
- *Feature Driven Development (FDD),*
- *Dynamic Systems Development Method (DSDM),*
- *Crystal Clear.*
- *экстремальное программирование (eXtreme Programming, XP).*

Артефакты для работы с требованиями в гибких методологиях

- **Карта представления системы** в определённой степени заменяет документ «концепция»;
- **Истории пользователей (user story)** сильно напоминают краткие описания вариантов использования.
- **Приёмочные** тесты обычно пишут на обратной стороне карты с соответствующей историей пользователя.
- **CRC-карты** (Класс-Ответственность-Кооперация). Заголовок и таблица в две колонки — ответственности (методы класса) и классы, имеющие связи с описываемым классом.

Гибкие технологии разработки ПО

- **Минимизируют риски** благодаря разделению процесса разработки на маленькие промежутки времени (*итерации*), обычно 1-4 недели.
- Каждая **итерация** может рассматриваться как полноценный проект (может включать в себя планирование, анализ требований, проектирование, реализацию, тестирование и документирование).
- Обычно результатом итерации не является продукт, готовый к выходу на рынок. Но целью каждой итерации является получение **стабильной версии продукта**.
- В конце каждой итерации происходит **переоценка приоритетов** проекта, что значительно сокращает риски.

Все гибкие методологии имеют общие характеристики:

- **итеративная разработка;**
- **фокус на взаимодействии** и коммуникации;
- полный или частичный **отказ от создания дорогостоящих промежуточных артефактов** проекта.

Гибкие технологии разработки программного обеспечения исходят из принципа, что только **10%** от ожидаемого в действительности понадобится в первоначальном виде, т.е. **90%** времени будет потрачено впустую.

Основные идеи agile

- Личности и их взаимодействие важнее, чем процессы и инструменты.
- Работающее программное обеспечение важнее, чем полная документация.
- Сотрудничество с заказчиком важнее, чем переговоры по контракту.
- Реакция на изменения важнее, чем следование плану.

Краеугольным камнем гибких технологий программирования является **разработка через тестирование**:

- автоматические тесты пишутся для любой части реализации, которая гипотетически «может сломаться»;
- тесты пишутся непосредственно *перед* написанием соответствующего кода;
- существующий код никогда не меняется без написания соответствующих тестов;
- выполняется регулярный запуск всех автоматических тестов.

Основы манифеста гибких технологий

- Главное – удовлетворение требований заказчика путем скорой и непрерывной поставки ценного и работоспособного ПО.
- Приветствуются изменяющиеся требования: их используют для повышения конкурентоспособности продукта.
- Работоспособное ПО поставляется как можно чаще, периодами от пары недель до пары месяцев.
- Бизнесмены и разработчики ежедневно работают сообща.
- Проекты строятся вокруг мотивированных личностей, которым оказывается доверие и создаются все условия для работы.
- Наиболее эффективным способом передачи информации (как внутри команды разработчиков, так и вовне) является личный разговор.
- Основной мерой прогресса является работоспособное ПО.
- Устанавливается удобный режим ведения разработки.
- Непрерывное внимание к техническому совершенству и хорошему дизайну повышает гибкость.
- Простота — искусство НЕ делать лишней работы.
- Лучшие архитектурные решения, наборы требований и дизайны создаются самоорганизующимися командами.
- Команда регулярно рассматривает и внедряет любые методы повышения своей эффективности.

Планирование на основе требований на примере XP

- Основная идея экстремального программирования (XP) — устранить высокую стоимость изменений, вносимых в ПО в процессе как разработки, так и эксплуатации.

Планирование включает следующие работы:

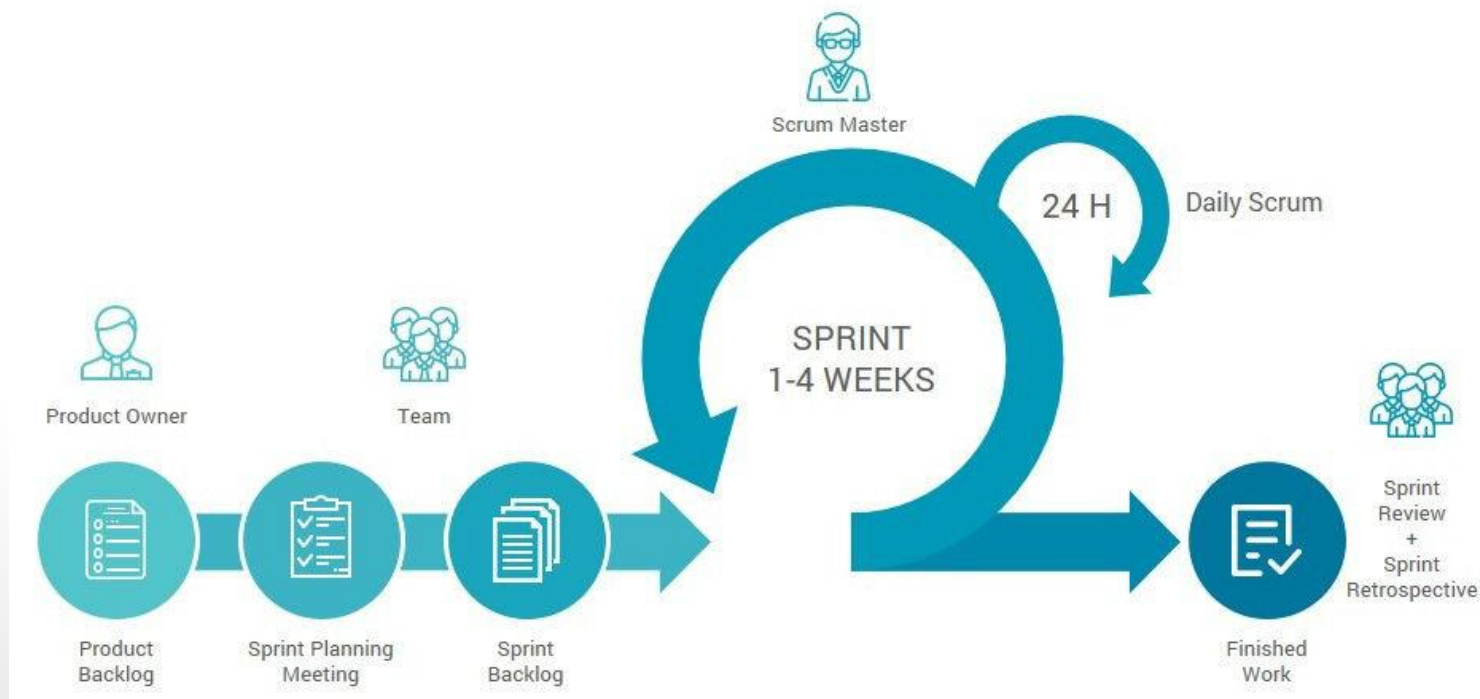
- оценивание,
- планирование версий и итераций.
- **Оценивание** представляет собой определение объёма работ в разрезе историй пользователя.
- **Планирование** в XP базируется на следующих основных понятиях:
 - *производительность, приоритеты, стоимость версии, составление плана версий, составление плана итераций.*

Основные идеи Scrum

- В настоящее время Scrum является одной из наиболее популярных методологий разработки ПО.
- Scrum – это каркас разработки, с использованием которого разработчики могут решать появляющиеся проблемы, при этом продуктивно производить продукты высочайшей значимости с точки зрения клиента.
- И Scrum, и XP (экстремальное программирование) требуют от команд завершения вполне осязаемого объема работы, который можно предоставить пользователю в конце каждой итерации.
- Эти итерации планируются таким образом, чтобы быть короткими и фиксированными по времени.

Основные идеи Scrum

Роли	Артефакты	Процессы
- Владелец продукта - Скрам-мастер - Команда	- Бэклог продукта - Бэклог спринта - Инкремент продукта	- Планирование спринта - Обзор спринта - Ретроспектива - Скрам-митинг - Спринт

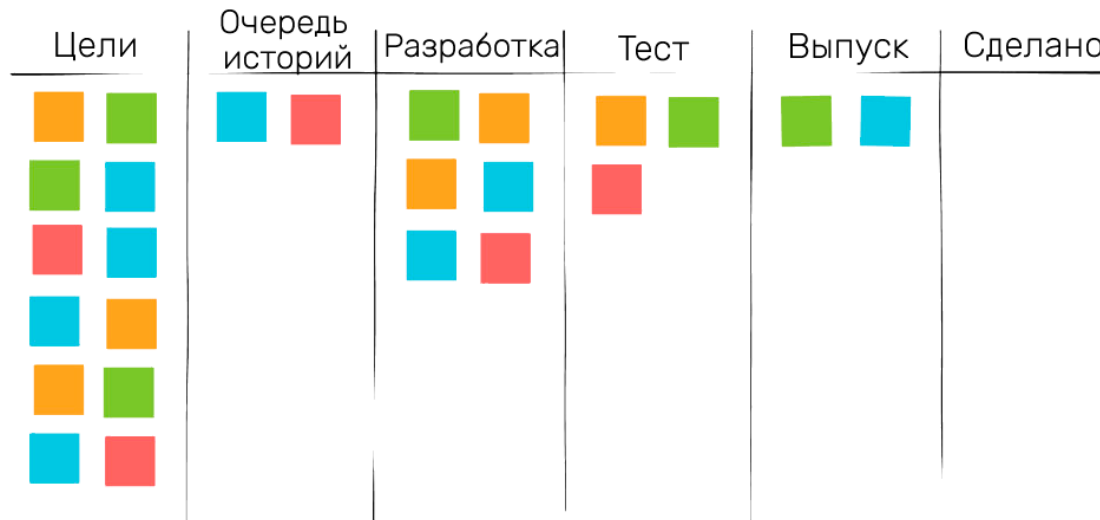


Основные идеи Kanban

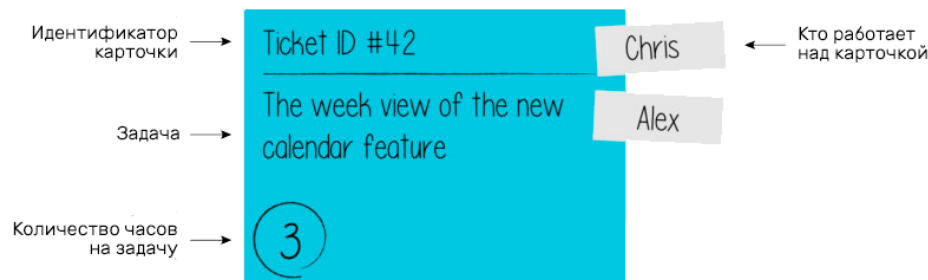
- Методология *Канбан* (Kanban) пытается ограничивать ненужную незаконченную работу так, чтобы она никогда не выходила за рамки пропускной способности команды.
- В данной методологии отменяется разработка по фазам с четкими временными границами, пользовательские истории становятся больше, а их самих меньше.
- Оценка результатов сводится к минимуму или исключается из цикла разработки. Внимание переходит со скорости разработки на продолжительность цикла.
- Канбан разработка строится вокруг *визуальной доски*, которая используется для управления незаконченной работой.

Основные идеи Kanban.

Визуальная доска



История (Green) Брак (Red) Задачи (Cyan) Свойство (Orange)



Основные идеи Kanban

Методологию Канбан можно описать всего тремя основными правилами:

- **Визуализируйте разработку.** Разделите работу на задачи, каждую задачу напишите на карточке и поместите на стену или доску. Используйте названные столбцы, чтобы показать положение задачи при разработке.
- **Ограничивайте WIP** (work in progress или работу, выполняемую одновременно) на каждом этапе разработки ПО.
- **Измеряйте время цикла** (среднее время на выполнение одной задачи) и постоянно оптимизируйте процесс, чтобы уменьшить это время.

Анализ требований и управление рисками

- **Риск** в реализации программных проектов – это потенциальная проблема, которая имеет существенную вероятность отрицательно повлиять на успешность проекта.
- **Управление риском** – комплекс мероприятий по выявлению, оценке, предотвращению и контролю рисков проекта.

Стратегии и работы по управлению риском



Действия по управлению рисками

- Работы по *оцениванию риска* (risk assessment) начинаются с определения потенциальных опасностей для проекта.
- *Анализ риска* сводится к исследованию и описанию потенциальных последствий конкретных факторов риска для проекта, а также вероятности их проявления.
- *Определение приоритетов* состоит из поиска ответов на два вопроса:
 - насколько вероятно проявление риска в проекте;
 - насколько разрушительны могут быть последствия его проявления.

Стратегии поведения в отношении рисков

- **Предотвращение риска** (risk avoidance) – это процесс реорганизации проекта таким образом, чтобы риск не мог на него воздействовать.
- **Передача риска** - перераспределение работ проекта таким образом, чтобы кто-то другой (Заказчик, партнёр и т.п.) отвечал за работу с ним.
- **Принятие риска** обязывает Разработчика «заботиться» о нём.
 - Мероприятия по контролю риска (*risk control*) включают планирование, разрешение и мониторинг.

Современные тенденции в развитии АИС

- развитие и появление новой элементной базы,
- появление и развитие интегрированных сред разработки,
- появление глобальных сетей передачи данных,
- глобализация бизнеса,
- переход к экономике, ориентированной на потребителя,
- появление и развитие электронного бизнеса и др.

Покупное или заказное ПО – критерии выбора

Выбор осуществляется даже не из 2, а из 3 вариантов:

1. закупить решение у вендора;
2. заказать эксклюзивное решение у фирмы-производителя прикладного программного обеспечения.
3. изготовить решение самостоятельно.

Стратегии выбора решения:

- анализ требований;
- анализ несоответствия;
- подход на основе «лучших практик».

Анализ требований

Практическое применение методов анализа требований, применительно к задаче выбора покупного решения, требует ответа на следующие вопросы:

- **рамки проекта;**
- **широта анализа требований.**
- **глубина проработки требований;**
- **требуемые ресурсы.**

Анализ несоответствия

- Анализ несоответствия при выборе КИС базируется на классических методах бизнес-анализа, предполагающих построения моделей «как есть», «как надо» и переходной модели, показывающей путь реформирования предприятия.

Подход на основе «лучших практик»

- Предлагается не останавливаться на существующих на предприятии бизнес-процессах, а запустить процесс реинжиниринга, основываясь на «лучших практиках» внедряемого ERP-решения.
1. Каждый пакет ERP соответствует нуждам организации.
 2. Копировать существующие процессы не имеет смысла.
 3. Априорный анализ лучших практик не должен использоваться.
 4. Следует реорганизовать бизнес на основе лучших практик так, чтобы изменения в ERP-системы были минимальны.

Процесс выбора решения

- Выбор решения для построения на предприятии корпоративной АИС – очень ответственный процесс, связанный с вопросами защиты инвестиций и выживания на рынке.

Поэтому задачу выбора решения следует рассматривать в рамках проектного и процессного подходов, со всеми вытекающими отсюда последствиями.

Процесс быстрого выбора для быстрого внедрения организован достаточно просто.

Он предусматривает выполнение следующих этапов:

- сформировать команду «выборщиков»;
- организовать демонстрацию КИС поставщиками;
- осуществить предварительный отбор;
- осуществить выбор.

Список литературы

1. Вигерс Карл Разработка требований к программному обеспечению/Пер, с англ. — М.:Издательско-торговый дом «Русская Редакция», 2004. —576с.: ил.
2. Леффингуелл Д., Уидриг Д. Принципы работы с требованиями к программному обеспечению. М.: ИД “Вильямс”, 2002.
3. Введение в Rational Unified Process/ Ф. Кратчен — СПб.: Вильямс, 2002. — 240 с.
4. Астелс, Дэвид; Миллер Гранвилл; Новак, Мирослав. Практическое руководство по экстремальному программированию.: Пер. с англ. — М.: Издательский дом «Вильямс», 2002. — 320 с.: ил. — Парал. тит. англ.
5. Бек К. Экстремальное программирование. — СПб.: Питер, 2002. — 224 с.
6. G.Chemis (1999), Shooting the Rapids of a Rapid Implementation //Shape Your World, Focus '99 (Denver, CO:J.D.Edwards)